Experiment 4
Classification of Discrete-Time Systems

Name: Corbin Ezekiel O. Gutierrez Section Code: CPE32 Date of Activity: 2/26/2026

Objectives:

By the end of this experiment, students should be able to:

- Implement discrete-time system equations in Python.
- Test whether a system is linear, time-invariant, causal, stable, memoryless, and invertible.
- Automate system property verification using Python functions.
- Interpret results and classify given systems.

Background Theory

A discrete-time system $y[n] = T\{x[n]\}$may exhibit several key properties:

• **Property**	• **Definition**
• **Linearity**	• Satisfies superposition: additivity + homogeneity
• **Time-Invariance (TI)**	• Time shifts do not change system behavior
• **Causality**	• Depends only on present/past inputs
• **Stability (BIBO)**	• Bounded input → bounded output
• **Memorylessness**	• Output depends only on current input
• **Invertibility**	• Different inputs must not produce the same output

Materials / Requirements

- PC or laptop

Software

- Python (NumPy + Matplotlib)
- Scilab

Experiment Tasks

You will analyze the following systems:

System A: $y[n] = 3x[n] - x[n-1]$
System B: $y[n] = x[n]^2$
System C: $y[n] = n \cdot x[n]$

Step 1 — Import Libraries and Define Test Signals

```python
import numpy as np

# Test signals
n = np.arange(-5, 6)
x1 = np.sin(0.2 * np.pi * n)
x2 = np.cos(0.4 * np.pi * n)

a = 2  # scaling factor

```

Step 2 — Define System Functions

```python
# System A: y[n] = 3x[n] - x[n-1]
def system_A(x):
    y = np.zeros_like(x)
    y[1:] = 3*x[1:] - x[:-1]
    y[0] = 3*x[0]  # assume x[-1] = 0
    return y

# System B: y[n] = x[n]^2
def system_B(x):
    return x**2

# System C: y[n] = n * x[n]
def system_C(x):
    return n * x

```

Part 1 — Linearity Test

A system is linear if: $\boldsymbol{T(ax_1 + x_2) = aT(x_1) + T(x_2)}$

```python
def test_linearity(system):
    lhs = system(a*x1 + x2)
    rhs = a*system(x1) + system(x2)
    return np.allclose(lhs, rhs)

print("System A Linear?", test_linearity(system_A))
print("System B Linear?", test_linearity(system_B))
print("System C Linear?", test_linearity(system_C))
```

Part 2 — Time-Invariance Test

A system is TI if: $\boldsymbol{T(x[n - n_0]) = y[n - n_0]}$

```python
shift = 2

def shift_signal(x, k):
    return np.roll(x, k)

def test_time_invariance(system):
    x_shifted = shift_signal(x1, shift)
    y1 = system(x1)
    y2 = system(x_shifted)
    y1_shifted = shift_signal(y1, shift)
    return np.allclose(y1_shifted, y2)

print("System A TI?", test_time_invariance(system_A))
print("System B TI?", test_time_invariance(system_B))
print("System C TI?", test_time_invariance(system_C))
```

Part 3 — Causality Test

A system is causal if output at time $n$does not depend on future input.

```
def test_causality(system):
    x = np.zeros(11)
    x[5] = 1  # impulse at n = 0 (centered)
    y = system(x)
    # if output before index 5 changes, it's non-causal
    return np.all(y[:5] == 0)

print("System A Causal?", test_causality(system_A))
print("System B Causal?", test_causality(system_B))
print("System C Causal?", test_causality(system_C))
``
```

Part 4 — Stability Test (BIBO)

Bounded input → bounded output.

```
def test_stability(system):
    x = np.ones(11) * 5    # bounded input
    y = system(x)
    return np.all(np.abs(y) < 1e6)

print("System A Stable?", test_stability(system_A))
print("System B Stable?", test_stability(system_B))
print("System C Stable?", test_stability(system_C))
```

Part 5 — Memorylessness Test

A system is memoryless if $y[n]$depends only on $x[n]$.

```
def test_memoryless(system):
    x = np.random.randn(11)
    y = system(x.copy())
    y_modified = system(np.roll(x, 1))
    # if shifting input changes output at other indices → has memory
    return np.allclose(y, y_modified)

print("System A Memoryless?", test_memoryless(system_A))
print("System B Memoryless?", test_memoryless(system_B))
print("System C Memoryless?", test_memoryless(system_C))
```

Part 6 — Invertibility Test

A system is invertible if different inputs never yield the same output.

```
def test_invertibility(system):
    xA = np.random.randn(11)
    xB = np.random.randn(11)
    yA = system(xA)
    yB = system(xB)
    # if outputs are same but inputs differ → non-invertible
    return not np.allclose(yA, yB)

print("System A Invertible?", test_invertibility(system_A))
print("System B Invertible?", test_invertibility(system_B))
print("System C Invertible?", test_invertibility(system_C))
``
```

```
l\.vscode\extensions\ms-pytho
' 'C:\Users\Ezekiel\DSP\act4\
Linearity Test
System A : True
System B : False
System C : True

Time-Invariance Test
System A : False
System B : True
System C : False

Causality Test
System A : True
System B : True
System C : True

Stability Test
System A : True
System B : True
System C : True

Memorylessness Test
System A : False
System B : False
System C : False

Invertibility Test
System A : True
System B : True
System C : True
PS C:\Users\Ezekiel\DSP>
```

OUTPUT TABLE (to fill in)

Put ✓ if the system follows the property, otherwise put ✕.

Property	System A	System B	System C
Linearity	✓	✕	✓
Time-Invariant	✕	✓	✕
Causal	✓	✓	✓
Stable	✓	✓	✓
Memoryless	✕	✕	✕
Invertible	✓	✓	✓

Guide Questions

1. **Which system(s) violated linearity? Why?**

   System B broke linearity because it squares the input, which changes how the output behaves.

2. **Why does multiplying by $n$make a system time-varying?**

   Multiplying by n makes the system change over time. The output is different depending on when the input happens.

3. **Which systems have memory and why?**

   System A has memory because it uses past input.

4. **Why is system $y[n] = x[n]^2$non-invertible?**

   You can't get back the input from the output. For example, $2^2 = 4$ and $(-2)^2 = 4$, so the output is the same for different inputs.

5. **Why is stability essential for signal processing systems like filters?**

   Stability makes sure small inputs don't cause big outputs. This keeps signals safe and clear.

Observation/Discussion/Conclusion:

In this experiment, we studied three systems and looked at their properties. System A is linear, time-invariant, causal, stable, and has memory because it uses past input values. System B is nonlinear, time-invariant, causal, stable, and memoryless since it only uses the current input. System C is linear, causal, time-varying, and unstable. We learned that squaring the input breaks linearity, multiplying by n makes a system time-varying, memory exists when a system uses past input, non-invertible systems can't recover the input from the output, and stability keeps outputs from becoming too large. This experiment helped us understand how these properties affect the way systems process signals.